to find rows that match the criteria of having given values in certain columns. For example, if an Active Record class called Book were mapping a relational table called books and the books table included columns called title and author, you used to be able write this (it's since been deprecated, but we kind of miss it):

```
pickaxe = Book.find_by_title("Programming Ruby")
daves_books = Book.find_all_by_author("Dave Thomas")
```

Active Record did not predefine all these potential finder methods. Instead, it uses our old friend method_missing. Inside that method, it looks for calls to undefined methods that match the pattern /^find_(all_)?by_(.*)/.It also looks for /^find_or_(initialize|create)_by_(.*)/. If the method being invoked does not match this pattern or if the fields in the method name don't correspond to columns in the database table, Active Record calls super so that a genuine method_missing report will be generated.

A Metaprogramming Example

Let's bring together all the metaprogramming topics we've discussed in a final example by writing a module that allows us to trace the execution of methods in any class that mixes the module in. This would let us write the following:

metaprogramming/trace_calls_example.rb

```
require_relative "trace_calls"

class Example
  def one(arg)
    puts "One called with #{arg}"
  end
end

ex1 = Example.new
ex1.one("Hello") # no tracing from this call

class Example
  include TraceCalls
  def two(arg1, arg2)
    arg1 + arg2
  end
end

ex1.one("Goodbye") # but we see tracing from these two
puts ex1.two(4, 5)
```

produces:

```
One called with Hello
==> calling one with ["Goodbye"]
One called with Goodbye
<== one returned nil
==> calling two with [4, 5]
<== two returned 9
9
```

We can see immediately that there's a subtlety here. When we mix the TraceCalls module into a class, it has to add tracing to any existing instance methods in that class. It also has to arrange to add tracing to any methods we subsequently add.

Let's start with the full listing of the TraceCalls module: